Vidyavardhini's

College of Engineering & Technology

Vasai Road (W)

Department of
Artificial Intelligence and Data Science

Laboratory Manual

Semester	VIII	Class	B.E.
Course Code	CSL801		
Course Name	Advanced Artificial Intelligence Lab		

Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education; always aiming at becoming a valuable resource for industry and society.

Mission

- To provide a technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.

Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9. Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12. Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Objectives

1	To introduce probabilistic models
2	To understand Generative Adversarial Networks (GANs)
3	To learn Variational Autoencoders (VAEs)
4	To understand the concept of Transfer Learning.
5	To explore Ensemble Learning techniques
6	To build intelligent applications

Course Outcomes

CO	Students will be able to	Action verbs	Bloom's Level
CSL801.1	Implement a Probabilistic Model for predicting outcomes	Implement	Apply (level 3)
CSL801.2	Apply a GAN on to solve the given problem	Apply	Apply (level 3)
CSL801.3	Apply appropriate Variational Autoencoder architecture to solve the given problem	Apply	Apply (level 3)
CSL801.4	Apply the concept of Transfer Learning on the given dataset	Apply	Apply (level 3)
CSL801.5	Apply Ensemble Learning techniques to real-world problems	Apply	Apply (level 3)
CSL801.6	Build an application by making use of Advanced AI concepts	Build	Create (level 6)

Mapping Experiments with Course Outcomes

PCL402: **Database Management System Lab**

List of Experiments	Course Outcomes					
	CSL801.1	CSL801.2	CSL801.3	CSL801.4	CSL801.5	CSL801.6
Prediction of Stock Market Trends Using Hidden Markov Model.	3	-	-	-	-	-
Perform Edge detection on the MNIST using Markov Random Field (MRFs)	3	-	-	-	-	-
Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset.	-	3	-	-	-	-
Train a Variational Autoencoder on Fashion MNIST.	-	-	3	-	-	-
Explore the working of any pre-trained model towards outcome generation.	-	-	-	3	-	-
Apply Random Forest algorithm for credit fraud detection	-	-	-	-	3	-
Case-study on the implementation of AI in Autonomous Vehicles.	-	-	-	-	-	3
Mini – Project	-	-	-	-	-	3

Sr. No.	Name of Experiment	D.O.P.	D.O.C.	Page No.	Remark

PCL402: **Database Management System Lab**

1	Prediction of Stock Market Trends Using Hidden Markov Model.				
2	Perform Edge detection on the MNIST using Markov Random Field (MRFs)				
3	Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset				
4	Train a Variational Autoencoder on Fashion MNIST.				
5	Explore the working of any pre-trained model towards outcome generation.				
6	Apply Random Forest algorithm for credit fraud detection.				
7	Case-study on the implementation of AI in Autonomous Vehicles.				
8	Mini – Project				

D.O.P: Date of performance

D.O.C : Date of correction

Experiment No. 1
Prediction of Stock Market Trends Using Hidden Markov Model.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Prediction of Stock Market Trends Using Hidden Markov Model.

Objective:

• To understand the concept of Hidden Markov Models (HMM).
• To analyze stock market data using probabilistic models.
• To predict hidden states such as market trends (bullish/bearish).
• To apply HMM for time-series forecasting.

Theory:

Hidden Markov Model (HMM):
A Hidden Markov Model is a statistical model used to represent systems that follow a Markov process with hidden (unobservable) states.
In stock market prediction:
The observed data = stock prices, returns, or volume
The hidden states = market conditions (bull market, bear market, stable)

Components of HMM:
States (S): Hidden states like Bullish, Bearish
Observations (O): Stock prices or returns
Transition Probability (A): Probability of moving from one state to another
Emission Probability (B): Probability of observation given a state
Initial Probability (π): Starting probability of states

Working of HMM in Stock Prediction:

The model learns patterns from historical stock data

It identifies hidden market states

Based on probabilities, it predicts future trends

Implementation:

• Collect historical stock price data.

• Preprocess data (calculate returns, normalize values).

• Define number of hidden states (e.g., 2 or 3 states).

• Train HMM model using algorithms like Baum-Welch.

• Predict hidden states using Viterbi algorithm.

• Analyze predicted states to determine market trend.

Code:

```
# Install required library first:
# pip install hmmlearn yfinance numpy pandas matplotlib

import numpy as np
import pandas as pd
import yfinance as yf
from hmmlearn.hmm import GaussianHMM
import matplotlib.pyplot as plt

# Step 1: Download stock data (example: TCS)
data = yf.download("TCS.NS", start="2020-01-01", end="2024-01-01")

# Step 2: Preprocess data (calculate daily returns)
data['Returns'] = data['Close'].pct_change()
data = data.dropna()

# Step 3: Prepare observations
X = data['Returns'].values.reshape(-1, 1)

# Step 4: Define and train HMM
model = GaussianHMM(n_components=2, covariance_type="full", n_iter=1000)
model.fit(X)

# Step 5: Predict hidden states
hidden_states = model.predict(X)

# Step 6: Add states to dataframe
```

```
data['State'] = hidden_states

# Step 7: Plot results
plt.figure(figsize=(12,6))

for i in range(model.n_components):
    state = (hidden_states == i)
    plt.plot(data.index[state], data['Close'][state], '.', label=f'State {i}')

plt.title("Stock Price with Hidden States")
plt.legend()
plt.show()
```

Conclusion:

Hidden Markov Models can effectively model stock market behavior by identifying hidden trends. They provide a probabilistic approach to predicting market conditions, though accuracy depends on data quality and model assumptions.

Experiment No. 2
Perform Edge detection on the MNIST using Markov Random Field (MRFs).
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Perform Edge detection on the MNIST using Markov Random Field (MRFs).

• To understand the concept of Markov Random Fields (MRFs).
• To apply MRF for image processing tasks.
• To perform edge detection on image datasets like MNIST.
• To analyze spatial dependencies between pixels.Objective:

Theory:

Markov Random Field (MRF):

A Markov Random Field is a probabilistic graphical model used to model spatial relationships in data. It is widely used in image processing where each pixel depends on its neighboring pixels.

Unlike Hidden Markov Models, MRFs are undirected graphical models, meaning relationships are mutual and not directional.

Application in Edge Detection: In images:

- Each pixel is considered a node
- Neighboring pixels influence each other
- Edges are detected based on intensity differences between neighboring pixels

MRF helps in:

- Smoothing images
- Detecting edges
- Reducing noise

Energy Function in MRF:

MRF works by minimizing an **energy function**, which consists of:

- **Data term:** Depends on observed pixel values
- **Smoothness term:** Depends on neighboring pixels

Implementation:

- Load MNIST dataset images.
- Preprocess images (grayscale normalization).
- Define neighborhood system (e.g., 4-neighbour or 8-neighbour).
- Construct MRF model representing pixel relationships.
- Define energy function based on pixel intensity differences.
- Apply optimization technique (e.g., Iterated Conditional Modes - ICM).
- Detect edges based on changes in pixel intensities.

Code:

```
# Install dependencies:

# pip install numpy matplotlib tensorflow


import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist


# Step 1: Load MNIST dataset

(x_train, _), (_, _) = mnist.load_data()


# Step 2: Select one image

image = x_train[0].astype(np.float32)


# Step 3: Normalize image

image = image / 255.0
```

```
# Step 4: Define neighborhood (4-neighbour)
def get_neighbors(i, j, shape):
    neighbors = []
    if i > 0: neighbors.append((i-1, j))
    if i < shape[0]-1: neighbors.append((i+1, j))
    if j > 0: neighbors.append((i, j-1))
    if j < shape[1]-1: neighbors.append((i, j+1))
    return neighbors

# Step 5: Initialize labels (0 = no edge, 1 = edge)
labels = np.zeros_like(image)

# Step 6: Define energy function
def energy(i, j, label):
    data_term = abs(image[i, j] - label)

    smoothness_term = 0
    for ni, nj in get_neighbors(i, j, image.shape):
        smoothness_term += abs(label - labels[ni, nj])

    return data_term + 0.5 * smoothness_term
```

```
# Step 7: Apply Iterated Conditional Modes (ICM)
for iteration in range(5):
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            e0 = energy(i, j, 0)
            e1 = energy(i, j, 1)
            labels[i, j] = 0 if e0 < e1 else 1

# Step 8: Display results
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')

plt.subplot(1,2,2)
plt.title("Detected Edges (MRF)")
plt.imshow(labels, cmap='gray')

plt.show()
```

Conclusion:

Markov Random Fields are effective for modeling spatial relationships in images. They help in detecting edges by considering neighboring pixel dependencies, making them useful in image processing tasks.

Experiment No. 3
Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset

Objective:

• To understand the concept of Generative Adversarial Networks (GANs).
• To implement Deep Convolutional GAN (DCGAN) for image generation.
• To train a model on MNIST dataset for generating handwritten digits.
• To analyze the performance of generator and discriminator networks.

Theory:

Generative Adversarial Network (GAN):
GAN is a deep learning model consisting of two neural networks:
Generator: Generates fake data (images)
Discriminator: Distinguishes between real and fake data
Both networks are trained simultaneously in a competitive manner.

Deep Convolutional GAN (DCGAN):

DCGAN is an extension of GAN that uses Convolutional Neural Networks (CNNs) instead of fully connected layers.

Key Features of DCGAN:

- Uses convolutional layers for image processing
- Uses batch normalization
- Uses ReLU (Generator) and LeakyReLU (Discriminator)
- Removes pooling layers

Working:

1. Generator creates fake images from random noise
2. Discriminator evaluates images (real or fake)
3. Both networks improve through training
4. Generator learns to create realistic images

Implementation:

• Import required libraries such as TensorFlow/Keras, NumPy, and Matplotlib.
• Load and preprocess MNIST dataset (normalize images).
• Define the Generator network using convolutional layers.
• Define the Discriminator network using CNN layers.
• Combine both networks to form the DCGAN model.
• Train the model using:
  - Adversarial training process
  - Binary Cross-Entropy loss function
• Generate new images using trained Generator.
• Visualize generated images.

Code:

```
# Install dependencies:

# pip install tensorflow matplotlib numpy


import tensorflow as tf

from tensorflow.keras import layers

import numpy as np

import matplotlib.pyplot as plt


# Step 1: Load and preprocess MNIST dataset

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()


x_train = x_train.astype('float32')

x_train = (x_train - 127.5) / 127.5   # Normalize to [-1, 1]

x_train = np.expand_dims(x_train, axis=-1)
```

```
BUFFER_SIZE = 60000

BATCH_SIZE = 256


train_dataset = tf.data.Dataset.from_tensor_slices(x_train)\

    .shuffle(BUFFER_SIZE).batch(BATCH_SIZE)


# Step 2: Build Generator

def build_generator():

    model = tf.keras.Sequential()


    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())


    model.add(layers.Reshape((7, 7, 256)))


    model.add(layers.Conv2DTranspose(128, (5,5), strides=(1,1), padding='same',
use_bias=False))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())
```

```
    model.add(layers.Conv2DTranspose(64, (5,5), strides=(2,2), padding='same',
use_bias=False))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())



    model.add(layers.Conv2DTranspose(1, (5,5), strides=(2,2), padding='same',
use_bias=False, activation='tanh'))



    return model



# Step 3: Build Discriminator

def build_discriminator():

    model = tf.keras.Sequential()



    model.add(layers.Conv2D(64, (5,5), strides=(2,2), padding='same',
input_shape=[28,28,1]))

    model.add(layers.LeakyReLU())

    model.add(layers.Dropout(0.3))



    model.add(layers.Conv2D(128, (5,5), strides=(2,2), padding='same'))

    model.add(layers.LeakyReLU())

    model.add(layers.Dropout(0.3))



    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(1))

    return model

generator = build_generator()
discriminator = build_discriminator()

# Step 4: Loss and Optimizers
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Step 5: Training
```

```
EPOCHS = 20

noise_dim = 100

num_examples_to_generate = 16


seed = tf.random.normal([num_examples_to_generate, noise_dim])


@tf.function

def train_step(images):

    noise = tf.random.normal([BATCH_SIZE, noise_dim])


    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise, training=True)


        real_output = discriminator(images, training=True)

        fake_output = discriminator(generated_images, training=True)


        gen_loss = generator_loss(fake_output)

        disc_loss = discriminator_loss(real_output, fake_output)


    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)

    gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)
```

```
    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))

    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))



def train(dataset, epochs):

    for epoch in range(epochs):

        for image_batch in dataset:

            train_step(image_batch)



        print(f"Epoch {epoch+1} completed")



        generate_and_save_images(generator, epoch + 1, seed)



# Step 6: Generate and display images

def generate_and_save_images(model, epoch, test_input):

    predictions = model(test_input, training=False)



    plt.figure(figsize=(4,4))



    for i in range(predictions.shape[0]):

        plt.subplot(4,4,i+1)

        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
```

```
    plt.axis('off')

    plt.suptitle(f"Epoch {epoch}")

    plt.show()


# Step 7: Train Model

train(train_dataset, EPOCHS)
```

Conclusion:

DCGAN successfully generates realistic handwritten digit images from the MNIST dataset. It demonstrates the power of generative models in deep learning and their applications in image synthesis.

Experiment No. 4
Train a Variational Autoencoder on Fashion MNIST.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Train a Variational Autoencoder on Fashion MNIST.

Objective:

• To understand the concept of Autoencoders and Variational Autoencoders (VAE).
• To implement VAE for image generation and representation learning.
• To train the model on Fashion MNIST dataset.
• To analyze latent space representation of images.

Theory:

Autoencoder:
An Autoencoder is a neural network used for learning efficient data representations. It consists of:
Encoder: Compresses input into latent representation
Decoder: Reconstructs input from latent representation

Variational Autoencoder (VAE):
VAE is a probabilistic version of an autoencoder that learns a distribution instead of fixed encoding.

Instead of mapping input to a single point, it maps to:

Mean (μ)

Variance (σ^2)

A latent variable is sampled from this distribution.

Key Concepts:
Latent Space: Compressed representation of data
Reparameterization Trick: Allows backpropagation through sampling
Loss Function:
Reconstruction Loss

KL Divergence Loss

Working:

1. Input image is encoded into mean and variance
2. Latent vector is sampled
3. Decoder reconstructs the image
4. Model learns to generate similar images

Implementation:

• Import required libraries such as TensorFlow/Keras and NumPy.

• Load and preprocess Fashion MNIST dataset (normalize images).

• Define Encoder network to generate mean and variance.

• Apply reparameterization trick to sample latent vector.

• Define Decoder network to reconstruct images.

• Define loss function:

Reconstruction loss

KL divergence

• Train the VAE model on dataset.

• Generate new images from latent space.

Code:

 # Install dependencies:

 # pip install tensorflow numpy matplotlib

import tensorflow as tf

from tensorflow.keras import layers, Model

```
import numpy as np

import matplotlib.pyplot as plt


# Step 1: Load and preprocess Fashion MNIST

(x_train, _), (x_test, _) = tf.keras.datasets.fashion_mnist.load_data()


x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


x_train = np.reshape(x_train, (-1, 28, 28, 1))

x_test = np.reshape(x_test, (-1, 28, 28, 1))


latent_dim = 2


# Step 2: Encoder

inputs = layers.Input(shape=(28, 28, 1))

x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(inputs)

x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)

x = layers.Flatten()(x)

x = layers.Dense(16, activation="relu")(x)


z_mean = layers.Dense(latent_dim, name="z_mean")(x)
```

```
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)


# Step 3: Reparameterization Trick

def sampling(args):

    z_mean, z_log_var = args

    epsilon = tf.random.normal(shape=(tf.shape(z_mean)[0], latent_dim))

    return z_mean + tf.exp(0.5 * z_log_var) * epsilon


z = layers.Lambda(sampling, output_shape=(latent_dim,))([z_mean, z_log_var])


encoder = Model(inputs, [z_mean, z_log_var, z], name="encoder")


# Step 4: Decoder

latent_inputs = layers.Input(shape=(latent_dim,))

x = layers.Dense(7*7*64, activation="relu")(latent_inputs)

x = layers.Reshape((7, 7, 64))(x)


x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(x)

x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)

outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)


decoder = Model(latent_inputs, outputs, name="decoder")
```

```
# Step 5: VAE Model
class VAE(Model):
    def __init__(self, encoder, decoder):
        super(VAE, self).__init__()
        self.encoder = encoder
        self.decoder = decoder

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)

            # Reconstruction Loss
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    tf.keras.losses.binary_crossentropy(data, reconstruction), axis=(1,2)
                )
            )

            # KL Divergence Loss
            kl_loss = -0.5 * tf.reduce_mean(
```

```
        tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1)
    )

    total_loss = reconstruction_loss + kl_loss

    grads = tape.gradient(total_loss, self.trainable_weights)
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

    return {
        "loss": total_loss,
        "reconstruction_loss": reconstruction_loss,
        "kl_loss": kl_loss,
    }

vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam())

# Step 6: Train Model
vae.fit(x_train, epochs=20, batch_size=128, validation_data=(x_test, None))

# Step 7: Generate Images
def generate_images():
```

```
    z_sample = np.random.normal(size=(16, latent_dim))

    generated = decoder.predict(z_sample)



    plt.figure(figsize=(4,4))

    for i in range(16):

        plt.subplot(4,4,i+1)

        plt.imshow(generated[i].reshape(28,28), cmap='gray')

        plt.axis('off')

    plt.show()


generate_images()
```

Conclusion:

Variational Autoencoders are powerful generative models that learn data distributions and generate new samples. They are useful for image generation, feature learning, and dimensionality reduction.

Experiment No. 5
Explore the working of any pre-trained model towards outcome generation.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Explore the working of any pre-trained model towards outcome generation.

Objective:

• To understand the concept of pre-trained models.
• To explore transfer learning in deep learning.
• To use a pre-trained model for prediction tasks.
• To analyze the performance of the model on given data.

Theory:

Pre-trained Models:

Pre-trained models are deep learning models that are already trained on large datasets (such as ImageNet). These models can be reused for solving similar problems without training from scratch.

Transfer Learning:

Transfer learning is a technique where knowledge gained from one task is applied to another related task. It helps in:

- Reducing training time
- Improving performance
- Working with smaller datasets

Examples of Pre-trained Models:

- VGG16
- ResNet
- MobileNet
- Inception

Working:

1. Load a pre-trained model

2. Use it directly or fine-tune it
3. Provide input data
4. Generate predictions

Implementation:

• Import required libraries such as TensorFlow/Keras.

• Load a pre-trained model (e.g., MobileNet or VGG16).

• Preprocess input data according to model requirements.

• Load and prepare dataset (images).

• Use the model to perform prediction.

• Analyze output results.

Code:

```
# Install dependencies:

# pip install tensorflow pillow matplotlib numpy


import tensorflow as tf

import numpy as np

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions

from tensorflow.keras.preprocessing import image

import matplotlib.pyplot as plt


# Step 1: Load pre-trained model

model = MobileNetV2(weights='imagenet')
```

```
# Step 2: Load and preprocess image

img_path = 'sample.jpg'  # replace with your image file

img = image.load_img(img_path, target_size=(224, 224))


plt.imshow(img)

plt.title("Input Image")

plt.axis('off')

plt.show()


x = image.img_to_array(img)

x = np.expand_dims(x, axis=0)

x = preprocess_input(x)


# Step 3: Make prediction

preds = model.predict(x)


# Step 4: Decode predictions

decoded = decode_predictions(preds, top=3)[0]


print("Top Predictions:")

for i, (imagenetID, label, prob) in enumerate(decoded):
```

```
print(f"{i+1}: {label} ({prob*100:.2f}%)")
```

Conclusion:

Pre-trained models are efficient for solving real-world problems. They reduce computational cost and training time while providing high accuracy through transfer learning.

Experiment No. 6
Apply Random Forest algorithm for credit fraud detection.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Apply Random Forest algorithm for credit fraud detection.

Objective:

• To understand the concept of Ensemble Learning.
• To implement Random Forest algorithm for classification.
• To detect fraudulent transactions using machine learning.
• To analyze model performance on real-world data.

Theory:

Credit Fraud Detection:

Credit fraud detection involves identifying fraudulent transactions from a large set of financial data. It is a classification problem where transactions are labeled as fraudulent or non-fraudulent.

Random Forest Algorithm:

Random Forest is an ensemble learning technique that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

Key Concepts:

Decision Trees: Base models used in Random Forest

Bagging (Bootstrap Aggregation): Random sampling of data

Feature Randomness: Random selection of features for each tree

Voting Mechanism: Final output based on majority voting

Advantages:

- High accuracy
- Handles large datasets
- Reduces overfitting
- Works well with missing data

Implementation:

• Import required libraries such as NumPy, Pandas, and Scikit-learn.

• Load the credit card transaction dataset.

• Preprocess data (handle missing values, normalize features).

• Split dataset into training and testing sets.

• Initialize Random Forest classifier.

• Train the model using training data.

• Predict results on test data.

• Evaluate model performance using accuracy, precision, and recall.

Code:

```
# Install dependencies:

# pip install pandas numpy scikit-learn matplotlib seaborn


import pandas as pd

import numpy as np

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report


# Step 1: Load dataset

# Replace with your dataset path (e.g., creditcard.csv from Kaggle)

data = pd.read_csv("creditcard.csv")
```

```
# Step 2: Explore dataset

print(data.head())

print(data['Class'].value_counts())  # 0 = Normal, 1 = Fraud


# Step 3: Handle imbalance (optional - sampling)

fraud = data[data['Class'] == 1]

normal = data[data['Class'] == 0].sample(len(fraud))


data_balanced = pd.concat([fraud, normal])


# Step 4: Split features and labels

X = data_balanced.drop('Class', axis=1)

y = data_balanced['Class']


# Step 5: Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Step 6: Initialize Random Forest

model = RandomForestClassifier(n_estimators=100, random_state=42)


# Step 7: Train model

model.fit(X_train, y_train)
```

```
# Step 8: Predictions

y_pred = model.predict(X_test)


# Step 9: Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred))


print("\nClassification Report:\n")

print(classification_report(y_test, y_pred))
```

Conclusion:

Random Forest is an effective algorithm for fraud detection. It improves prediction accuracy by combining multiple decision trees and helps in identifying fraudulent transactions efficiently.

Experiment No. 7
Case-study on the implementation of AI in Autonomous Vehicles.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Case-study on the implementation of AI in Autonomous Vehicles.

Objective:

• To understand the role of Artificial Intelligence in autonomous vehicles.
• To study different AI techniques used in self-driving cars.
• To analyze real-world applications of AI in transportation systems.
• To explore challenges and future scope of autonomous vehicles.

Theory:

Autonomous Vehicles:

Autonomous vehicles (self-driving cars) are vehicles that can operate without human intervention using Artificial Intelligence, sensors, and advanced algorithms.

Role of AI in Autonomous Vehicles:

AI enables vehicles to:
Perceive surroundings
Make decisions
Navigate safely

Key Technologies Used:

Computer Vision: Used for object detection (cars, pedestrians, traffic signals).

Machine Learning / Deep Learning: Used for decision-making and prediction.

Sensor Fusion: Combines data from: Cameras, LiDAR, Radar

Path Planning Algorithms: Determines safe route and movement.

Control Systems: Controls steering, braking, and acceleration.

Working of Autonomous Vehicles:

- Sensors collect environmental data
- AI processes and interprets data
- System identifies objects and obstacles
- Decision-making algorithms determine actions
- Vehicle executes movement safely

Case Study: Tesla, Inc. Autonomous Driving System

One of the most prominent real-world implementations of AI in autonomous vehicles is developed by Tesla, Inc..

Tesla Autopilot System:

Tesla uses an advanced Autopilot system that enables semi-autonomous driving using AI.

Working of Tesla AI System:

- **Tesla vehicles are equipped with multiple cameras, ultrasonic sensors, and radar.**
- **The system uses computer vision and deep learning models to detect:**
  - **Vehicles**
  - **Pedestrians**
  - **Traffic lights and road signs**
- **AI models process real-time data to understand surroundings and predict risks.**
- **Neural networks are trained using large-scale data collected globally.**
- **Based on predictions, the system performs:**
  - **Lane keeping**
  - **Automatic braking**
  - **Adaptive cruise control**
- **The system improves continuously through over-the-air updates.**

Outcome:

The implementation of AI in Tesla vehicles results in:

- **Reduced driver workload**
- **Improved road safety**
- **Enhanced driving efficiency**

- **Semi-autonomous driving capabilities**

Challenges in Autonomous Vehicles:

- **Safety and reliability concerns**
- **High computational requirements**
- **Ethical decision-making (e.g., accident scenarios)**
- **Adverse weather conditions affecting sensors**
- **Legal and regulatory issues**

Future Scope:

- **Fully autonomous (Level 5) vehicles**
- **Smart traffic systems integration**
- **AI-powered transportation networks**
- **Reduced accidents and improved mobility**

Conclusion:

Artificial Intelligence plays a crucial role in autonomous vehicles by enabling perception, decision-making, and control. Systems like those developed by Tesla, Inc. demonstrate the real-world impact of AI in transportation. While challenges remain, AI-driven vehicles have the potential to revolutionize the future of mobility.

PCL402: **Database Management System Lab**

Experiment No. 8
Mini – Project
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim:

To develop a machine translation system that translates text from multiple languages into Gujarati using a fine-tuned AI language model.

Objective:

- **To apply advanced AI concepts in Natural Language Processing (NLP).**
- **To build a multilingual translation system.**
- **To fine-tune a pre-trained language model for Gujarati translation.**
- **To evaluate translation performance using standard metrics.**

Theory:

Machine Translation:

Machine Translation (MT) is a subfield of Natural Language Processing that focuses on automatically translating text from one language to another.

Pre-trained Language Models:

Modern translation systems use transformer-based models such as:

- **BERT**
- **GPT**
- **T5**

These models are trained on large datasets and can be fine-tuned for specific tasks.

Fine-Tuning:

Fine-tuning is the process of adapting a pre-trained model to a specific task using a smaller, task-specific dataset.

Tokenization:

Text is converted into tokens (subwords/words) before feeding into the model.
A custom tokenizer can be trained for Gujarati to improve translation quality.

Transformer Architecture:

Transformers use attention mechanisms to understand context and relationships between words, making them highly effective for translation tasks.

Project Overview:

The project focuses on building a multilingual-to-Gujarati translation system using a 1 Billion parameter fine-tuned model.

Implementation:

Step 1: Dataset Collection

- **Collected multilingual parallel datasets (English–Gujarati, Hindi–Gujarati).**
- **Sources: Open datasets like OPUS, AI4Bharat.**

Step 2: Data Preprocessing

- **Cleaned text (removed special characters, noise)**
- **Normalized Gujarati script**
- **Tokenized sentences**

Step 3: Tokenizer Training

```
from tokenizers import ByteLevelBPETokenizer


tokenizer = ByteLevelBPETokenizer()

tokenizer.train(files=["gujarati_corpus.txt"], vocab_size=30000, min_frequency=2)
```

```
tokenizer.save_model("gujarati_tokenizer")
```

Step 4: Model Selection

- **Used transformer-based model (like T5 / GPT architecture)**
- **Loaded pre-trained weights**

Step 5: Fine-Tuning Model

```
from transformers import T5ForConditionalGeneration, T5Tokenizer, Trainer,
TrainingArguments



model = T5ForConditionalGeneration.from_pretrained("t5-small")

tokenizer = T5Tokenizer.from_pretrained("t5-small")



def preprocess(example):

    input_text = "translate to gujarati: " + example['source']

    target_text = example['target']



    model_input = tokenizer(input_text, truncation=True, padding='max_length',
max_length=128)

    labels = tokenizer(target_text, truncation=True, padding='max_length',
max_length=128)



    model_input["labels"] = labels["input_ids"]

    return model_input



# Training arguments
```

```
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset
)

trainer.train()
```

Step 6: Translation Function

```
def translate(text):
    input_text = "translate to gujarati: " + text
    inputs = tokenizer.encode(input_text, return_tensors="pt")

    outputs = model.generate(inputs, max_length=100)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

print(translate("Hello, how are you?"))
```

Step 7: Evaluation Metrics

- **BLEU Score (translation quality)**
- **Accuracy (approximate matching)**
- **Human evaluation for fluency**

Result:

- **The model successfully translated text from English/Hindi to Gujarati.**
- **Fine-tuning improved translation accuracy significantly.**
- **The system handled simple and moderate sentences effectively.**

Applications:

- **Regional language translation systems**
- **Chatbots in Gujarati**
- **Government and educational tools**
- **Content localization**

Challenges Faced:

- **Limited Gujarati datasets**
- **Handling grammar and sentence structure**
- **Training large models requires high computation**
- **Translation errors in complex sentences**

Future Scope:

- **Improve model with larger datasets**
- **Support more languages (multilingual system)**
- **Deploy as web/mobile application**
- **Use larger models (1B+ parameters for better accuracy)**

Conclusion:

This mini project demonstrates the practical application of AI in natural language processing. The developed system successfully translates multilingual text into Gujarati using a fine-tuned transformer model. It highlights the importance of transfer learning and showcases how AI can be used to bridge language barriers.